CodeMarker

Qualitative and mixed-methods data analysis inside Obsidian.

CodeMarker is a suite of 7 Obsidian plugins for coding, annotating, and analyzing qualitative data across 6 source types — Markdown, CSV, PDF, images, audio, and video — with a unified codebook and a consolidated analytics module featuring 10 analysis engines and 17 visualization modes.

~35,000 lines of TypeScript. ~4,600 lines of CSS. One shared codebook. Zero proprietary formats.

Why

Qualitative research tools cost $800–1,200, lock your data in proprietary formats, and force you into a separate application disconnected from the rest of your workflow.

CodeMarker runs where you already think and write. Your codes, memos, and analytical structures live as plain JSON in your Obsidian vault — versionable with Git, syncable with any file service, yours to keep.

The Suite

Plugin	Source	Engine	LOC
codemarker-v2	Markdown	CodeMirror 6	5,300 TS + 717 CSS
codemarker-csv	Tabular data	AG Grid + CM6	8,200 TS + 951 CSS
codemarker-pdf	Documents	PDF.js (Obsidian native)	5,200 TS + 534 CSS
codemarker-image	Images	Fabric.js	2,800 TS + 564 CSS
codemarker-audio	Audio	WaveSurfer.js v7	2,650 TS + 561 CSS
codemarker-video	Video	HTML5 video + WaveSurfer.js	2,680 TS + 579 CSS
codemarker-analytics	Cross-plugin analysis	Chart.js + svd-js	7,900 TS + 730 CSS

Create a code in any plugin — it's immediately available in all others via the shared registry.

Source Coding

📝 Markdown

Full CM6 engine: colored highlights, drag-resizable handles, a margin panel with brackets, labels and hover

tooltips — modeled after MAXQDA. Smart layering resolves nested and overlapping markers. Selection menu for creating markers, hover menu for editing existing ones.

📊 CSV

The largest plugin. Opens CSV files as native AG Grid views with sort, filter, and resize. Two marker types: **RowMarker** (entire row) and **SegmentMarker** (text selection within a cell). Clicking a cell opens a full CM6 editor with all the Markdown coding features. Header-level tagging applies codes across all rows at once.

📄 PDF

Codes directly on Obsidian's native PDF view — no external dependencies. Two coding modes: **text selection** (character-level precision via text layer indices) and **shape drawing** (rectangle, circle, freehand polygon overlays). Margin panel rendered outside the scroll container so labels are never clipped. Compatible with Obsidian v1.7.7 and v1.8.0+.

🖼️ Image

Draw regions on images — rectangles, ellipses, or freeform polygons. All coordinates are normalized (0–1) relative to natural image dimensions, so codes survive resizing. Fabric.js canvas with glow highlights and rendered labels over shapes.

🎵 Audio

WaveSurfer.js waveform with drag-to-select temporal coding. Colored regions with vertical lane stacking for overlaps. Minimap overview with marker position bars. Transport controls for playback, zoom, speed, and volume. Bidirectional hover between waveform regions and sidebar.

🎬 Video

Video player on top, synchronized waveform below — WaveSurfer.js receives the `<video>` element as a media source and renders the audio track's waveform automatically. Frame capture exports the current moment as PNG. Same region engine as audio.

Analytics

The analytics module reads `data.json` from all 6 coding plugins and provides **10 analysis engines** across **17 visualization modes**.

Engines

Engine	What it computes
Frequency	Code counts with breakdown by source type and file
Co-occurrence	Code × code matrix — absolute, percentage, Jaccard, Dice, or presence. Three sort modes: alphabetical, frequency, hierarchical cluster

Engine	What it computes
Document-Code Matrix	Document × code frequency matrix — input for clustering, MCA, MDS, and decision trees
Evolution	Code distribution by normalized position within documents (spatial coding patterns)
Temporal	Cumulative code frequency over real time (chronological coding patterns)
Text Stats	Word count, unique words, type-token ratio, average words and characters per segment
Lag Sequential Analysis	Conditional transition probabilities between codes at lag N with adjusted z-scores. Bakeman & Quera (Ch.12 Routledge)
Cluster Analysis	Agglomerative hierarchical clustering with average linkage, dendrogram, cut-line, and silhouette scores. Rousseeuw 1987 (Ch.5 Routledge)
MCA	Multiple Correspondence Analysis — biplot via SVD of indicator matrix. Code and marker positions in 2D with explained inertia
MDS	Classical Multidimensional Scaling (Torgerson) — Jaccard distance projection for codes or files with Kruskal stress-1
Decision Tree	CHAID-style chi-square recursive splitting with accuracy, Klecka's tau, and error analysis linked to text retrieval (Ch.6 + Ch.16 Routledge)
Word Frequency	Tokenized word cloud with PT/EN stopword filtering and code association
Text Extractor	Cross-source text extraction — character-level for Markdown/CSV/PDF, temporal labels for audio/video, with file caching

Views

#	View	Description
1	**Dashboard**	KPI cards + clickable thumbnails of every view
2	**Frequency**	Bar chart with source breakdown
3	**Co-occurrence**	Heatmap with 5 display modes
4	**Graph**	Force-directed co-occurrence network with edge labels
5	**Doc-Matrix**	Document × code heatmap
6	**Evolution**	Positional scatter plot per file
7	**Temporal**	Cumulative line chart over time

#	View	Description
8	**Text Retrieval**	Coded segment cards, grouped by code or file, with search
9	**Word Cloud**	Weighted word cloud with stopword config
10	**Text Stats**	Lexical metrics table per code
11	**MCA**	2D biplot with toggle markers/labels and inertia
12	**MDS**	2D projection for codes or files with stress indicator
13	**Dendrogram**	Hierarchical tree with cut-line and silhouette
14	**Lag Sequential**	Transition heatmap at configurable lag
15	**Polar Coords**	Activation/inhibition quadrant map (prospective/retrospective)
16	**Chi-square**	Independence test between codes and categorical variables
17	**Decision Tree**	CHAID tree with accuracy metrics and error text retrieval

Every view shares a config panel: source filters, code filters, minimum frequency threshold, and view-specific parameters.

Data Model

Position models by source

Source	Position	Example
Markdown	{ from, to }	Character offsets in text
CSV segment	{ row, column, from, to }	Cell + character offsets
CSV row	{ row }	Entire row
Image	{ region }	Normalized 0–1 coordinates
PDF text	{ page, from, to }	Page + text layer offsets
PDF shape	{ page, region }	Page + shape coordinates
Audio	{ from, to }	Seconds (float)
Video	{ from, to }	Seconds (float)

Unified marker (Analytics)

All markers normalize into a single type for cross-source analysis:

```typescript
interface UnifiedMarker {
  id: string;
  source: "markdown" | "csv-segment" | "csv-row" | "image" | "pdf" | "audio" | "video";
  file: string;
  codes: string[];
  meta?: {
    row?, column?,          // CSV
    regionType?,            // Image
    fromLine?, toLine?,     // Markdown
    page?, pdfText?,        // PDF
    audioFrom?, audioTo?,   // Audio
    videoFrom?, videoTo?,   // Video
    createdAt?              // Timestamp
  };
}
```

Shared codebook

All plugins sync through `.obsidian/codemarker-shared/registry.json`:

```typescript
interface CodeDefinition {
  name: string;
  color: string;
  description?: string;
  paletteIndex: number;
  createdAt: number;
  updatedAt: number;
}
```

Merge strategy: highest `updatedAt` wins. Create a code while coding audio — it appears in your PDF, CSV, image, and analytics views instantly.

Storage

```
your-vault/
├── .obsidian/
│   ├── codemarker-shared/
│   │   └── registry.json          ← shared codebook
```

```
│   └── plugins/
│       ├── codemarker-v2/data.json    ← markdown markers
│       ├── codemarker-csv/data.json   ← csv markers
│       ├── codemarker-pdf/data.json   ← pdf markers
│       ├── codemarker-image/data.json ← image markers
│       ├── codemarker-audio/data.json ← audio markers
│       └── codemarker-video/data.json ← video markers
```

Plain JSON. No database. No cloud. Back up with a copy, version with Git, sync however you want.

Academic Foundations

The analytics architecture is grounded in the **Routledge Reviewer's Guide to Mixed Methods Analysis** (2021, Onwuegbuzie & Johnson):

Chapter	Technique	Status
Ch.1	Inter-respondent matrix, quantitizing	✅ Doc-Code Matrix, Frequency
Ch.5	Cluster Analysis — Jaccard, dendrogram, silhouette	✅ Complete
Ch.6	CHAID decision tree	✅ Complete
Ch.12	Diachronic Analysis — LSA, Polar Coords	✅ Complete
Ch.14	Qualitizing — Magnitude Coding	⬜ Planned
Ch.16	Ethnographic Decision Models	✅ Decision Tree view
Ch.23	Joint Display	⬜ Planned
Ch.24	Case Comparison Table	🟡 CSV viewer as partial
Ch.2	Factor Analysis Topic Modeling	⬜ Planned
Ch.7	Multiple Linear Regression	💡 Conceptual

A Python prototype (~12,400 LOC) served as an NLP playground informing future features: sentiment analysis, contradiction detection, topic modeling via LDA, concept networks, and narrative block detection.

Comparison

	CodeMarker	MAXQDA	Atlas.ti	NVivo
Text coding	✅	✅	✅	✅
Image coding	✅	✅	✅	✅
PDF coding (text + shapes)	✅	✅	✅	✅
Audio coding	✅	✅	✅	✅
Video coding	✅	✅	✅	✅
CSV/tabular coding	✅	❌	❌	❌
Co-occurrence analysis	✅	✅	✅	✅
MCA biplot	✅	✅	❌	❌
MDS projection	✅	❌	❌	❌
Hierarchical clustering + silhouette	✅	❌	❌	❌
Lag Sequential Analysis	✅	❌	❌	❌
CHAID decision tree	✅	❌	❌	❌
Polar coordinates	✅	❌	❌	❌
Word cloud	✅	✅	✅	✅
Runs inside your editor	✅	❌	❌	❌
Plain JSON storage	✅	❌	❌	❌
Git-friendly	✅	❌	❌	❌
Linked to your PKM	✅	❌	❌	❌
Open source	✅	❌	❌	❌
Price	Free	~€1,000	~€800	~€1,200

Getting Started

Installation

Each plugin installs independently — use only what you need.

```bash
cd /path/to/vault/.obsidian/plugins/codemarker-[plugin]
npm install
npm run build
```

Restart Obsidian → Settings → Community Plugins → Enable.

The shared registry syncs automatically on first run.

Basic workflow

1. **Open a file** — any supported format in Obsidian's file explorer
2. **Select a segment** — drag text, draw a shape, select a time range
3. **Assign codes** — type a name or toggle existing codes in the popover
4. **Explore** — open the Code Explorer sidebar to see your coding tree
5. **Analyze** — open CodeMarker Analytics for cross-source insights

Source badges

Each source type has a distinct badge color across the suite:

Source	Badge	Color
Markdown	MD	#42A5F5
CSV Segment	CSV	#66BB6A
CSV Row	ROW	#81C784
Image	IMG	#FFA726
PDF	PDF	#EF5350
Audio	AUD	#AB47BC
Video	VID	#00ACC1

Roadmap

Implemented: 16/38 features (42%)

Near-term

- Promote to Note — turn coded segments into Obsidian notes
- Emergency export — REFI-QDA XML + CSV fallback
- Variable categorization wizard for CSV
- Code hierarchy and grouping
- Memo system across all plugins

Medium-term

- Cross-source queries
- NLP enrichment (sentiment, entities, themes)
- Audio/video transcription sync (Whisper)
- Review queue for systematic coding
- Topic modeling (LDA)
- Joint Display integration

Long-term

- VS Code extension (same core, different host)
- Community codebook sharing
- Real-time collaboration via CRDT
- REFI-QDA interoperability

Development

```bash
npm install          # dependencies
npm run build        # production build → main.js
npm run dev          # watch mode
npm run typecheck    # tsc --noEmit
```

Technical stack

Plugin	Dependencies
Markdown	CodeMirror 6 (via Obsidian)

Plugin	Dependencies
CSV	AG Grid Community v33+, PapaParse
PDF	Zero deps (PDF.js via Obsidian)
Image	Fabric.js 6.6.1
Audio	WaveSurfer.js v7
Video	WaveSurfer.js v7
Analytics	Chart.js (lazy), svd-js

Conventions

- CSS namespaced per plugin (`codemarker-audio-*`, `codemarker-video-*`, etc.)
- Never `registerExtensions` — always `active-leaf-change` interceptor
- Shared registry: bidirectional merge by `updatedAt`
- Never break the marker interface — Analytics reads `data.json` directly
- Position data uses each source's native coordinate system

Numbers

Metric	Value
Plugins	7
TypeScript	~35,000 LOC
CSS	~4,600 LOC
Analysis engines	10
View modes	17
Source types	7
Academic chapters studied	10
Features implemented	16/38 (42%)
Features in progress	3/38 (8%)
Features planned	12/38 (32%)

License

[TBD]

Built by Marlon Lemes.

Qualitative research tools shouldn't cost a thousand dollars, lock your data in proprietary formats, or force you to leave the environment where you already think and write.